SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High) Assessment
Tool Weightage:40%

Dr

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

I D DEVENDRA *(192525203) certify that this submission is my original work and that I I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:D DEVENDRA

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1 – Query Execution Plan and Heuristic Optimization

Problem Understanding

Given a SQL query with joins and selection conditions, the objective is to draw its execution plan and optimize it using heuristic rules. The important goal is to reduce intermediate results before expensive joins.

Sample Query

```
SELECT S.Name, D.DeptName
FROM Student S
JOIN Department D ON S.DeptNo = D.DeptNo
WHERE S.Age > 20
AND D.DeptName = 'CSE';
```

Constraint Analysis

1. Only students with Age > 20 are required.
2. Only the CSE department is required.
3. Unnecessary columns should be removed.
4. Selections should be performed before joins.
5. Joins should operate on smaller intermediate relations.

Initial Execution Plan

Student + Department

↓

JOIN on DeptNo

↓

FILTER Age > 20 and DeptName = CSE

↓

Final Projection: Name, DeptName

Optimized Execution Plan

Student

↓

Selection: Age > 20

↓

Projection: Name, DeptNo

↓

JOIN with filtered Department

↓

Projection: Name, DeptName

Solution Development

The optimized relational algebra is approximately:
π(Name, DeptName) [(π(Name, DeptNo)(σ(Age>20)(Student))) JOIN (π(DeptNo, DeptName)(σ(DeptName='CSE')(Department)))].

Application of Concepts

Selection pushdown, projection pushdown, and join ordering are applied to reduce the number of tuples and attributes processed by the join.

Justification

Filtering and projecting before the join reduces the size of intermediate relations. This decreases memory use and join processing cost, making the optimized execution plan more efficient.

Question 2 – Cost-Based Join Optimization

Problem Understanding

Choose among nested-loop join, sort-merge join, and hash join using relation sizes and page counts.

Assumptions

R = 1,000 tuples; S = 5,000 tuples; 100 tuples/page. Therefore B(R)=10 pages and B(S)=50 pages.

Constraint Analysis

The decision depends on page counts, tuple counts, available memory, sorting overhead, and whether the join condition is suitable for hashing.

Cost-Based Decision Flow

Input relation sizes and page counts

↓

Estimate Nested-Loop cost

↓

Estimate Sort-Merge cost

↓

Estimate Hash-Join cost

↓

Compare estimated costs

↓

Select lowest-cost feasible join

Solution Development

Nested-loop approximate cost:
B(R) + B(R)×B(S) = 10 + (10×50) = 510 page accesses.

Hash join basic partition/read estimate:
2×(B(R)+B(S)) = 2×(10+50) = 120 page accesses.

Sort-merge requires sorting and merging, so it introduces additional I/O compared with the simple hash estimate.

Join Method	Approx. Cost / Nature	Decision
Nested Loop	≈ 510 page accesses	Not preferred
Sort-Merge	Sorting + merging overhead	Moderate
Hash Join	≈ 120 page accesses	Preferred

Justification

For an equality join with suitable relation sizes, hash join provides the lowest estimated cost under the stated assumptions. The conclusion is based on the assumed values, since the assessment sheet does not provide specific numerical inputs.

Question 3 – JDBC Connectivity using PreparedStatement

Problem Understanding

Implement Java-to-MySQL connectivity and execute a parameterized query using PreparedStatement.

Constraints

Requires a MySQL database, JDBC driver, database URL, username/password, Connection, PreparedStatement, parameter binding, ResultSet processing, and resource closing.

Database Setup

```
CREATE DATABASE college;
USE college;

CREATE TABLE Student(
id INT PRIMARY KEY,
name VARCHAR(50),
dept VARCHAR(30)
);
```

Java Implementation

```
import java.sql.*;

public class JDBCExample {
public static void main(String[] args) {
String url = "jdbc:mysql://localhost:3306/college";
String user = "root";
```

```
String password = "root";

String sql = "SELECT * FROM Student WHERE dept = ?";

try {
Connection con = DriverManager.getConnection(url, user, password);
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, "CSE");

ResultSet rs = ps.executeQuery();

while (rs.next()) {
System.out.println(
rs.getInt("id") + " " +
rs.getString("name") + " " +
rs.getString("dept")
);
}

rs.close();
ps.close();
con.close();
} catch (SQLException e) {
e.printStackTrace();
}
}
}
```

JDBC Flow

Java Application

↓

JDBC Driver

↓

MySQL Connection

↓

PreparedStatement

↓

Execute Parameterized Query

↓

ResultSet

↓

Display Output

Justification

PreparedStatement separates SQL structure from parameter values, supports parameterized execution, and is safer than constructing SQL by string concatenation.

Question 4 – Conflict Serializability

Problem Understanding

Determine whether a schedule of concurrent transactions is conflict-serializable by constructing a precedence graph.

Sample Schedule

R1(A), W1(A), R2(A), W2(B), R3(B), W3(B), R4(A), W4(A)

Constraint Analysis

A conflict exists when two different transactions access the same data item and at least one operation is a write.

Precedence Graph Construction

Identify conflicting operations

↓

Create edge Ti → Tj for earlier conflicting operation

↓

Draw all transaction nodes

↓

Check graph for cycles

↓

Acyclic → Conflict Serializable

Solution Development

Relevant precedence edges for the sample are T1→T2, T2→T3, and T1→T4. The graph is acyclic.

Sample Precedence Graph

T1

↓

T2 and T4

↓

T3

Decision

The schedule is conflict-serializable. One compatible serial order is T1 → T2 → T3 → T4.

Justification

A schedule is conflict-serializable when its precedence graph has no cycle. Since the sample graph is acyclic, the schedule is conflict-serializable.

Question 5 – Basic Two-Phase Locking (2PL)

Problem Understanding

Apply Basic 2PL to concurrent transactions and identify a possible deadlock.

Sample Transactions

T1: Read(A), Write(A), Read(B)
T2: Read(B), Write(B), Read(A)

Constraint Analysis

Shared locks are used for reads, exclusive locks for writes. During the growing phase locks may be acquired; during the shrinking phase locks are released and no new locks may be acquired.

Basic 2PL

Growing Phase

↓

Acquire S/X locks

↓

Perform reads/writes

↓

Shrinking Phase

↓

Release locks

↓

Transaction completes

Sample Locking

T1: LOCK-X(A), W1(A), LOCK-S(B), R1(B), then unlock.
T2: LOCK-X(B), W2(B), LOCK-S(A), R2(A), then unlock.

Deadlock Analysis

If T1 holds A and waits for B while T2 holds B and waits for A, a circular wait occurs: T1 ↔ T2.

Deadlock Resolution

T1 holds A; T2 holds B

↓

T1 waits for B; T2 waits for A

↓

Detect circular wait

↓

Abort one transaction, e.g. T2

↓

Release B

↓

T1 continues and commits

↓

Restart T2

Justification

Basic 2PL guarantees conflict serializability. Aborting one transaction breaks the circular wait and permits progress.

Question 6 – Wait-Die and Wound-Wait

Problem Understanding

Apply timestamp-based deadlock prevention. Assume T1 timestamp=10 (older) and T2 timestamp=20 (younger).

Wait-Die Rule

Older requests younger's lock

↓

Older waits

↓

Younger requests older's lock

↓

Younger aborts (dies)

Wound-Wait Rule

Older requests younger's lock

↓

Younger aborts (wounded)

↓

Younger requests older's lock

↓

Younger waits

Situation	Wait-Die	Wound-Wait
Older → Younger	Older waits	Younger aborts
Younger → Older	Younger aborts	Younger waits

Justification

Both schemes prevent circular waiting by enforcing timestamp ordering. Wait-Die is non-preemptive, while Wound-Wait is preemptive.

Question 7 – Immediate Update Recovery

Problem Understanding

Use a transaction log with a checkpoint to recover after failure. Immediate update may write database changes before commit, so recovery can require both UNDO and REDO.

Sample Log

<START T1>
<T1, A, 100, 150>
<START T2>
<T2, B, 200, 250>
<COMMIT T1>
<CHECKPOINT>
<T2, C, 300, 350>
<SYSTEM FAILURE>

Constraint Analysis

T1 is committed, while T2 has no commit record. Therefore T1's effects must be retained and T2's effects must be removed.

Immediate Update Recovery

System Failure

↓

Locate checkpoint

↓

Identify committed transactions

↓

REDO committed T1

↓

Identify uncommitted T2

↓

UNDO T2

↓

Database recovered

Recovery Result

T1 → REDO.
T2 → UNDO.
For T2, B is restored from 250 to 200 and C from 350 to 300.

Justification

Immediate Update requires UNDO for incomplete transactions and REDO for committed changes when necessary.

Question 8 – Deferred Update (NO-UNDO/REDO)

Problem Understanding

Apply deferred update recovery to committed and uncommitted transactions.

Sample Log

<START T1>
<T1, A, 100, 150>
<START T2>
<T2, B, 200, 250>
<COMMIT T1>
<START T3>
<T3, C, 300, 350>
<SYSTEM FAILURE>

Constraint Analysis

In deferred update, uncommitted changes have not been written to the database. Therefore recovery does not need UNDO.

Deferred Update Recovery

System Failure

↓

Scan log

↓

Find committed T1

↓

REDO T1

↓

Find uncommitted T2/T3

↓

Ignore T2/T3

↓

Database recovered

Recovery Table

Transaction	Status	Recovery Action
T1	Committed	REDO
T2	Uncommitted	Ignore
T3	Uncommitted	Ignore

Justification

Deferred update follows NO-UNDO/REDO because only committed transactions have database effects that need to be redone.

Question 9 – Strict 2PL for Banking System

Problem Understanding

T1 transfers ₹1,000 from account A to B while T2 reads account A. The system must prevent T2 from seeing an intermediate balance.

Initial State

A = ₹10,000; B = ₹5,000.

Constraint Analysis

The banking system requires consistency, isolation, no dirty reads, and atomic transfer of funds. T1 must hold exclusive locks until commit.

Strict 2PL Banking Flow

T1 requests X-lock(A) and X-lock(B)

↓

T1 subtracts ₹1,000 from A

↓

T1 adds ₹1,000 to B

↓

T1 COMMIT

↓

T1 releases locks

↓

T2 obtains S-lock(A)

↓

T2 reads consistent balance

Strict 2PL Pseudocode

T1:
LOCK-X(A)
LOCK-X(B)
A = A - 1000
B = B + 1000
COMMIT
UNLOCK(A), UNLOCK(B)

T2:
LOCK-S(A)
READ(A)
COMMIT
UNLOCK(A)

Final Balances

A = ₹9,000; B = ₹6,000.

Justification

T2 waits until T1 commits and releases the exclusive lock. Therefore T2 reads a consistent final balance rather than an intermediate transfer state.

Question 10 – Heuristic Query Optimization Pseudocode

Problem Understanding

Write pseudocode for query optimization using selection pushdown, early projection, and join ordering by smaller intermediate results.

Sample Query

```
SELECT S.Name, D.DeptName, C.CourseName
FROM Student S
JOIN Department D ON S.DeptNo = D.DeptNo
JOIN Course C ON S.CourseID = C.CourseID
WHERE S.Age > 20
AND D.DeptName = 'CSE';
```

Heuristic Optimization Flow

Parse SQL Query

↓

Build relational algebra tree

↓

Push selections down

↓

Push projections down

↓

Estimate intermediate sizes

↓

Join smaller relations first

↓

Build optimized plan

↓

Return optimized plan

Pseudocode

Algorithm HeuristicQueryOptimization(Query Q)

1. Parse Q into a relational algebra tree.
2. Identify all selection conditions.
3. Push each selection close to its base relation.
4. Identify attributes needed in the final result and join conditions.
5. Push projections toward the base relations.
6. Estimate intermediate relation sizes.
7. Arrange joins so smaller intermediate relations are generated first.
8. Construct the optimized query tree.
9. Return the optimized query plan.

End Algorithm

Application of Concepts

Selection pushdown removes unwanted tuples early. Projection pushdown removes unwanted attributes early. Join ordering reduces the size of intermediate results.

Justification

These heuristics reduce tuples, attributes, memory requirements, and join processing. Hence the resulting plan is generally more efficient than the unoptimized plan.